WhatNexus Project Analysis & AI Agents Implementation Guide

Project Scope Analysis

WhatNexus Overview

WhatNexus is a full-stack web application with a **multi-tenant architecture** consisting of:

Backend (whatnexus-backend/)

- **Node.js/Express** application (`src/app.js`)
- **Database layer** with models and services
- **Configuration management** via `.env`
- **Middleware stack** for request handling
- **Public assets** (HTML templates, images)
- **Utility functions** for common operations

Frontend (Whatnexus-frontend/)

- **Next.js 14+** application (modern React framework)
- **TypeScript** for type safety
- **Redux** for state management
- **Component-based UI** architecture
- **Custom hooks** for logic reuse
- **Services layer** for API communication
- **Helper utilities** and type definitions

Current Architecture Characteristics

- Multi-tenant SaaS platform
- Decoupled frontend/backend
- RESTful API communication
- Responsive UI with modern tooling (PostCSS, ESLint)

AI Agent Implementation Strategy

Phase 1: Architecture Foundation

1.1 Agent System Design

```
┌──────────────────────────────────────┐
│         AI Agent Manager             │
│  (Handles routing & orchestration)   │
└──────────────┬───────────────────────┘
               │
     ┌─────────┼─────────┬─────────────┐
     ▼         ▼         ▼             ▼
┌─────────┐ ┌─────────┐ ┌─────────┐  ┌─────────┐
│Tenant 1│ │Tenant 2│ │Tenant 3│  │Tenant N│
│Agent   │ │Agent   │ │Agent   │  │Agent   │
└─────────┘ └─────────┘ └─────────┘  └─────────┘
```

1.2 Technology Stack for AI Agents

- **LLM Framework**: LangChain.js or Vercel AI SDK
- **LLM Provider**: OpenAI, Anthropic, or open-source (Ollama)
- **Vector Store**: Pinecone, Weaviate, or Postgres with pgvector
- **Memory System**: Redis or in-memory cache
- **Message Queue**: Bull or RabbitMQ (for async tasks)

Phase 2: Step-by-Step Implementation

Step 1: Backend AI Service Setup

Create a new AI service module:

```
// filepath: whatnexus-backend/src/services/aiAgent.js
const { initializeAgentExecutor } = require("langchain/agents");
const { OpenAI } = require("langchain/llms");

class TenantAIAgent {
  constructor(tenantId, config) {
    this.tenantId = tenantId;
    this.llm = new OpenAI({
      apiKey: config.openaiKey,
      temperature: 0.7,
    });
    this.memory = new Map(); // Tenant-specific memory
    this.tools = [];
  }

  async initialize() {
    // Load tenant-specific context, documents, fine-tuned model
    await this.loadTenantContext();
    await this.setupTools();
  }

  async loadTenantContext() {
    // Fetch tenant data, docs, preferences from database
    // Store in vector database for RAG
  }

  async setupTools() {
    // Define tools specific to tenant needs
    // Tools could include: data queries, APIs, calculations
  }

  async executeQuery(userQuery) {
    // Process user query through agent
    // Return contextual response
  }
}

module.exports = TenantAIAgent;
```

Step 2: Tenant-Specific Configuration

```
// filepath: whatnexus-backend/src/config/aiAgentConfig.js
const tenantAgentConfigs = {
  tenant1: {
    name: "Tenant 1 AI Assistant",
    model: "gpt-4",
    systemPrompt: "You are an AI assistant specialized in...",
    tools: ["search", "calculate", "retrieve_data"],
    knowledgeBase: "tenant1_docs",
  },
  tenant2: {
    name: "Tenant 2 AI Assistant",
    model: "gpt-3.5-turbo",
    systemPrompt: "You are specialized in...",
    tools: ["analyze", "report", "query"],
    knowledgeBase: "tenant2_docs",
  },
};

module.exports = tenantAgentConfigs;
```

Step 3: Vector Store & RAG Setup

```
// filepath: whatnexus-backend/src/services/vectorStore.js
const { PineconeStore } = require("langchain/vectorstores");
const { OpenAIEmbeddings } = require("langchain/embeddings");

class TenantKnowledgeBase {
  constructor(tenantId) {
    this.tenantId = tenantId;
    this.embeddings = new OpenAIEmbeddings();
  }

  async ingestDocuments(documents) {
    // Store tenant documents in vector database
    // Documents indexed by tenantId + metadata
  }

  async retrieveContext(query, limit = 5) {
    // Retrieve relevant documents for query
    // Filter by tenantId for isolation
  }
}

module.exports = TenantKnowledgeBase;
```

Step 4: API Endpoints for Agent Interaction

```
// filepath: whatnexus-backend/src/routes/aiAgent.js
const express = require("express");
const router = express.Router();
const TenantAIAgent = require("../services/aiAgent");
const authMiddleware = require("../middlewares/auth");

// Initialize agent per tenant
const agentInstances = new Map();

async function getTenantAgent(tenantId) {
  if (!agentInstances.has(tenantId)) {
    const agent = new TenantAIAgent(tenantId, {
      /* config */
    });
    await agent.initialize();
    agentInstances.set(tenantId, agent);
  }
  return agentInstances.get(tenantId);
}

router.post("/query", authMiddleware, async (req, res) => {
  const { tenantId } = req.user;
  const { query } = req.body;

  try {
    const agent = await getTenantAgent(tenantId);
    const response = await agent.executeQuery(query);
    res.json({ success: true, response });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.post("/ingest-documents", authMiddleware, async (req, res) => {
  const { tenantId } = req.user;
  const { documents } = req.body;

  try {
    const kb = new TenantKnowledgeBase(tenantId);
    await kb.ingestDocuments(documents);
    res.json({ success: true });
```

```
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

Step 5: Frontend Integration

```
// filepath: Whatnexus-frontend/services/aiAgentService.ts
import axios from "axios";

class AIAgentService {
  private baseURL = process.env.NEXT_PUBLIC_API_URL;

  async queryAgent(query: string): Promise<string> {
    const response = await axios.post(
      `${this.baseURL}/api/ai-agent/query`,
      { query },
      { headers: { Authorization: `Bearer ${localStorage.getItem("token")}` } },
    );
    return response.data.response;
  }

  async ingestDocuments(documents: Document[]): Promise<void> {
    await axios.post(
      `${this.baseURL}/api/ai-agent/ingest-documents`,
      { documents },
      { headers: { Authorization: `Bearer ${localStorage.getItem("token")}` } },
    );
  }
}

export default new AIAgentService();
```

Step 6: Frontend UI Component

```
// filepath: Whatnexus-frontend/components/AIAssistant.tsx
"use client";

import { useState } from "react";
import aiAgentService from "@/services/aiAgentService";

export default function AIAssistant() {
  const [query, setQuery] = useState("");
  const [response, setResponse] = useState("");
  const [loading, setLoading] = useState(false);

  const handleQuery = async () => {
    setLoading(true);
    try {
      const result = await aiAgentService.queryAgent(query);
      setResponse(result);
    } catch (error) {
      console.error("Agent error:", error);
    }
    setLoading(false);
  };

  return (
    <div className="ai-assistant">
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Ask me anything..."
        disabled={loading}
      />
      <button onClick={handleQuery} disabled={loading}>
        {loading ? "Thinking..." : "Ask"}
      </button>
      {response && <div className="response">{response}</div>}
    </div>
  );
}
```

Step 7: Database Schema for Agent Data

```sql
-- Tenant AI Agent Configurations
CREATE TABLE tenant_ai_agents (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  agent_name VARCHAR(255),
  model_name VARCHAR(50),
  system_prompt TEXT,
  configuration JSONB,
  created_at TIMESTAMP,
  updated_at TIMESTAMP,
  FOREIGN KEY (tenant_id) REFERENCES tenants(id)
);

-- Agent Conversation History
CREATE TABLE agent_conversations (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  agent_id UUID NOT NULL,
  user_id UUID NOT NULL,
  query TEXT,
  response TEXT,
  metadata JSONB,
  created_at TIMESTAMP,
  FOREIGN KEY (tenant_id) REFERENCES tenants(id),
  FOREIGN KEY (agent_id) REFERENCES tenant_ai_agents(id)
);

-- Knowledge Base Documents
CREATE TABLE knowledge_base_documents (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  document_name VARCHAR(255),
  content TEXT,
  embedding_id VARCHAR(255),
  metadata JSONB,
  created_at TIMESTAMP,
  FOREIGN KEY (tenant_id) REFERENCES tenants(id)
);
```

Step 8: Environment Configuration

```
# filepath: whatnexus-backend/.env
OPENAI_API_KEY=your_api_key
PINECONE_API_KEY=your_key
PINECONE_ENVIRONMENT=your_env
REDIS_URL=redis://localhost:6379
AI_AGENT_MODEL=gpt-4
AI_AGENT_TEMPERATURE=0.7
```

Phase 3: Advanced Features

3.1 Multi-Agent Orchestration

- Route queries to specialized agents based on intent
- Parallel processing for complex queries
- Fallback mechanisms

3.2 Fine-Tuning Per Tenant

- Train models on tenant-specific data
- Custom embeddings for domain accuracy
- Continuous learning from interactions

3.3 Memory Management

- **Short-term**: Conversation context
- **Long-term**: Persistent knowledge base
- **Semantic caching**: For performance optimization

3.4 Monitoring & Analytics

- Track agent performance metrics
- Monitor token usage costs
- Tenant-specific usage reports

Phase 4: Deployment & Scaling

1. **Containerization**: Docker for agent services
2. **Orchestration**: Kubernetes for multi-tenant agents
3. **Load Balancing**: Distribute agent requests
4. **Monitoring**: Prometheus + Grafana
5. **Backup & Recovery**: Regular backups of knowledge bases

Implementation Timeline

Phase	Duration	Tasks
Setup	1-2 weeks	Framework selection, infrastructure setup
Core Agent	2-3 weeks	Basic agent creation, RAG integration
Multi-Tenant	2-3 weeks	Isolation, per-tenant configuration
Frontend	1-2 weeks	UI components, API integration
Testing	1-2 weeks	Unit tests, integration tests, QA
Deployment	1 week	Docker, monitoring, production setup

Total Estimated: 8-13 weeks

Key Considerations

- **Data Security**: Tenant isolation at all levels
- **Cost Management**: Monitor and optimize LLM API calls
- **Performance**: Cache responses, use async processing
- **Accuracy**: Regular evaluation and fine-tuning
- **Scalability**: Design for 100+ concurrent agents
- **Maintenance**: Automated monitoring and alerting

This roadmap provides a structured approach to implementing tenant-specific AI agents with production-ready architecture.